

SIMULAÇÃO DA PROVA PARCIAL 1

Universidade Federal de Mato Grosso do Sul Faculdade de Computação Verificação, Validação & Teste de Software 2026/1 Prof. Dr. Awdren Fontão	RGA: Nome completo:
--	----------------------------

(1) as respostas às questões só serão consideradas corretas se possuírem completude e conformidade, portanto, muita atenção ao enunciado; (2) a prova é de consulta e INDIVIDUAL. Qualquer interação com colegas fará a prova ser anulada; (3) o atraso para iniciar a prova será tolerado em, no máximo, 15 minutos; (4) o tempo máximo de duração é de 2 horas.

(Questão 1 – 2,5 pt) Uma startup está desenvolvendo a primeira versão de um sistema web para agendamento de consultas. O time é formado por profissionais recém-formados e ainda não possui experiência consolidada com práticas de qualidade. O primeiro módulo a ser entregue é o de cadastro e autenticação de pacientes. Durante uma conversa da equipe, surgiram as seguintes preocupações:

- “Precisamos garantir que o que foi implementado está de acordo com os requisitos.”
- “Precisamos saber se o sistema realmente atende ao que a clínica precisa no uso real.”
- “Precisamos executar o sistema para encontrar problemas antes da entrega.”
- Um integrante afirmou: “Vou depurar o sistema para revelar a presença de falhas.”

Responda:

A. Defina verificação, validação e teste de software no contexto do cenário apresentado.

Verificação: consiste em avaliar se o produto de software foi construído corretamente em relação aos artefatos produzidos, como requisitos, especificações, modelos e projeto. No cenário, envolve verificar se o módulo de cadastro e autenticação de pacientes foi implementado conforme os requisitos definidos.

Validação: consiste em avaliar se o software atende às necessidades reais dos usuários e do cliente, isto é, se o produto certo está sendo construído. No cenário, envolve analisar se o sistema de cadastro e autenticação realmente atende ao uso esperado pela clínica e pelos pacientes.

Teste de software: é uma atividade de VVT que envolve a execução do software com o objetivo de revelar falhas observáveis ou, quando falhas não são encontradas, aumentar a confiança sobre o comportamento do sistema.

B. Considerando o perfil da equipe e o momento do projeto, indique duas ações de VVT que você priorizaria agora e justifique.

Respostas possíveis incluem, desde que estejam coerentes e justificadas:

- **Revisão dos requisitos e dos fluxos de cadastro/autenticação**, para identificar ambiguidades, omissões e inconsistências antes ou durante a implementação.
- **Testes funcionais básicos dos fluxos principais**, cobrindo cadastro válido, cadastro inválido, login válido, login inválido, campos obrigatórios vazios e mensagens de erro.
- **Walkthrough do protótipo ou dos fluxos com stakeholders**, para validar se o comportamento esperado está alinhado ao uso real da clínica.
- **Checklist de qualidade dos requisitos**, especialmente por se tratar de uma equipe iniciante.

A justificativa deve considerar que o time é novato, o módulo é inicial e crítico para acesso ao sistema, e há necessidade de identificar defeitos simples cedo, antes que eles se propaguem para outras partes do produto.

C. Analise a fala do integrante sobre depuração. Ela está correta ou incorreta? Justifique.

Incorreta.

O correto seria dizer que o engenheiro está testando para revelar a presença de falhas. Depuração ocorre depois que um problema foi revelado: é a atividade de localizar e corrigir o defeito. A disciplina explicita que “depuração não é teste”.

D. Explique porque, nesse módulo, não seria viável depender de teste exaustivo.

Não seria viável depender de teste exaustivo porque, na prática, não é possível executar todas as combinações possíveis de entradas, estados, fluxos, perfis de usuário, dados inválidos, tentativas de autenticação e condições de uso.

Mesmo em um módulo aparentemente simples, como cadastro e autenticação, há muitas variações possíveis: campos vazios, formatos inválidos, senhas incorretas, usuários já cadastrados, falhas de conexão, tentativas repetidas de login, entre outras. Por isso, a estratégia de teste deve selecionar casos representativos, priorizando riscos, regras de negócio, valores críticos e comportamentos mais relevantes.

(Questão 2 – 2,5 pt) Uma squad experiente mantém um aplicativo de delivery com frontend web, app mobile, backend, banco de dados e integração com gateway de pagamento. O time já possui testes automatizados, pipeline de CI e deploy frequente, mas começou a enfrentar dois problemas: muitos testes E2E estão quebrando por pequenas mudanças na interface; bugs de integração ainda escapam para homologação.

A liderança pediu que o time reavaliasse sua estratégia de testes para equilibrar melhor: velocidade de feedback; custo de manutenção; cobertura de riscos; confiança para liberar novas versões.

Responda:

A. Explique por que equipes experientes ainda precisam de uma estratégia de testes e não apenas de “mais testes”;

Equipes experientes precisam de uma estratégia de testes porque qualidade não depende apenas da quantidade de testes, mas da capacidade de decidir quais testes fazer, em que nível, com qual objetivo e a que custo.

Uma boa estratégia considera trade-offs entre velocidade de feedback, custo de manutenção, cobertura de riscos, estabilidade dos testes e confiança para liberar novas versões. No cenário, simplesmente aumentar o número de testes E2E poderia piorar o problema, pois esses testes já estão frágeis diante de pequenas mudanças na interface.

B. Escolha um modelo de referência estudado na disciplina para apoiar essa decisão estratégica e explique como ele ajudaria nesse contexto;

A resposta mais esperada é a pirâmide de testes.

A pirâmide de testes ajuda a distribuir os testes em diferentes níveis, equilibrando custo, velocidade de feedback, facilidade de diagnóstico e cobertura de riscos. Ela sugere uma base maior de testes próximos do código, como testes unitários, uma camada intermediária de

testes de integração ou contrato, e uma quantidade menor de testes E2E, que costumam ser mais lentos, caros e frágeis.

No cenário, esse modelo ajuda a reduzir a dependência excessiva de testes E2E sensíveis a mudanças de interface e, ao mesmo tempo, reforçar os testes nos pontos onde bugs estão escapando, especialmente nas integrações entre frontend, app mobile, backend, banco de dados e gateway de pagamento.

Outros modelos podem ser aceitos se bem justificados, por exemplo: Árvore de testes, se o aluno discutir interfaces ricas, testes visuais, análise estática e redução de E2E frágeis. Modelo do queijo suíço, se o aluno argumentar sobre camadas complementares de defesa contra defeitos. Quadrantes de teste, se o aluno discutir equilíbrio entre testes técnicos, de negócio, automatizados, manuais, exploratórios e de aceitação.

- C. Com base no cenário, proponha uma redistribuição mais adequada dos esforços de teste entre níveis ou tipos de teste;

A resposta deve mostrar que o time está tentando equilibrar velocidade, custo de manutenção e cobertura dos riscos mais relevantes.

reduzir dependência excessiva de E2E;

aumentar testes unitários para regras de negócio;

reforçar testes de integração/contrato para comunicação entre frontend, backend, banco e gateway;

manter poucos E2E para fluxos realmente críticos;

complementar com testes UI/visuais ou smoke para reduzir custo de manutenção.

- D. Aponte uma limitação do modelo escolhido.

A pirâmide de testes é uma heurística, não uma regra universal. Ela orienta a distribuição dos testes, mas não decide automaticamente o que deve ser testado.

Sua aplicação depende do contexto do sistema, da arquitetura, dos riscos de negócio, da maturidade da equipe, do tipo de produto e dos custos de manutenção. Em alguns sistemas, como aplicações com interfaces muito ricas, integrações críticas ou forte dependência de serviços externos, pode ser necessário adaptar a pirâmide ou combiná-la com outros modelos.

(Questão 3 – 2,5 pt) Um time está especificando um aplicativo de mensagens instantâneas. Antes da implementação, foram produzidos: casos de uso do envio de mensagens; requisitos funcionais; protótipos de alta fidelidade.

Na análise inicial, surgiram dúvidas como: o que acontece se o contato estiver offline no momento do envio; o que acontece se o usuário tentar criar um grupo sem nome; se deveria existir a opção de desfazer a exclusão de uma mensagem por alguns segundos.

O time é novato, o prazo é curto, e você quer identificar omissões, ambiguidades, inconsistências e melhorias ainda nessa fase do projeto. As abordagens consideradas são:

1. revisão informal entre colegas;
2. walkthrough com leitura guiada do artefato;
3. inspeção formal com papéis definidos;
4. revisão orientada por checklist.

Responda:

- A. Diferencie revisão, walkthrough e inspeção.

Revisão: é uma atividade em que um artefato de software é examinado por pessoas do projeto ou partes interessadas, com o objetivo de identificar problemas, fornecer comentários, propor melhorias ou apoiar sua aprovação. É uma categoria ampla que pode incluir diferentes tipos de revisão.

Walkthrough: é um tipo de revisão geralmente mais leve e menos formal, em que o autor ou facilitador conduz os participantes pela leitura do artefato. É útil para promover entendimento compartilhado, esclarecer dúvidas e identificar lacunas, ambiguidades, inconsistências e melhorias, especialmente em requisitos, casos de uso, modelos e protótipos.

Inspeção: é uma revisão mais formal e sistemática, com papéis definidos, preparação prévia, critérios de análise, registro de discrepâncias ou defeitos e, quando necessário, reinspeção. É mais indicada quando se busca maior rigor, rastreabilidade e confiabilidade na análise do artefato.

- B. Qual abordagem você escolheria como principal neste momento? Justifique com base no perfil da equipe, no tipo de artefato e no objetivo da atividade.

As respostas mais esperadas são walkthrough ou revisão orientada por checklist.

Uma resposta adequada seria:

Como o time é novato, o prazo é curto e os artefatos analisados são requisitos, casos de uso e protótipos, uma abordagem mais leve e pedagógica tende a ser mais adequada neste momento. O walkthrough favorece a leitura guiada, a comunicação entre os membros do time e o entendimento comum sobre o comportamento esperado do sistema.

A revisão orientada por checklist também seria adequada, pois ajuda a direcionar a análise e reduz a chance de a equipe iniciante deixar passar problemas recorrentes, como omissões, ambiguidades, inconsistências e fluxos alternativos não especificados.

Uma boa resposta pode combinar as duas abordagens: realizar um walkthrough apoiado por checklist.

- C. Cite dois tipos de problema que essa atividade pode revelar nos requisitos ou protótipos;

Esperado:

- inconsistência;
- lacunas/omissões;
- ambiguidades;
- possíveis melhorias.

A aula de walkthrough usa exatamente exemplos como contato offline, grupo sem nome e opção de desfazer exclusão. A taxonomia de requisitos também lista omissão, ambiguidade e inconsistência.

- D. Em que situação você elevaria essa atividade de uma revisão mais leve para uma inspeção mais formal?

quando o artefato é crítico para segurança, confiabilidade ou conformidade;
quando há alta chance de impacto grave por defeitos;
quando o sistema exige rastreabilidade e rigor;
quando revisões leves anteriores mostraram muitos problemas;

quando se quer registrar defeitos de forma sistemática e decidir sobre reinspeção.

(Questão 4 – 2,5 pt) Você atua na homologação de uma nova versão de um app de delivery. Um dos requisitos definidos pelo time de produto é: “O valor total do pedido deve estar entre R\$ 10,00 e R\$ 500,00, inclusive, para que a finalização seja permitida.” Você terá acesso apenas à especificação e ao comportamento observável do sistema, sem analisar o código-fonte.

Responda:

- A. Escreva um caso de teste funcional completo para esse requisito, contendo ao menos: ID; objetivo; entradas; pré-condições; passos; resultado esperado.

ID: CT-FIN-01

Objetivo: verificar se o sistema permite finalizar pedidos com valor total dentro do intervalo de R\$10,00 a R\$500,00, inclusive.

Entradas: valor total do pedido = R\$10,00

Pré-condições:

- usuário autenticado no app;
- carrinho com itens adicionados;
- sistema disponível;
- fluxo de finalização acessível.

Passos:

1. Acessar o carrinho do app.
2. Adicionar itens até totalizar R\$10,00.
3. Prosseguir para a finalização do pedido.
4. Confirmar a tentativa de concluir a compra.

Resultado esperado:

- o sistema deve permitir a finalização do pedido;
- não deve exibir mensagem de erro relacionada ao valor mínimo;
- o pedido deve seguir para a próxima etapa/ser confirmado conforme o fluxo esperado.

A definição esperada de caso de teste é: descrição de um teste específico com valores de entrada, pré-condições/restrições, procedimentos e resultados esperados.

- B. Explique por que esse caso de teste é classificado como funcional / caixa-preta.

Porque o caso foi derivado da especificação do requisito, observando apenas entradas e saídas esperadas, sem considerar a implementação interna ou o código-fonte. A técnica funcional trata o sistema como caixa-preta.

- C. Indique quatro valores de entrada relevantes para testar esse requisito, incluindo valores de borda, e justifique brevemente.

Resposta esperada:

- **R\$9,99** ou valor abaixo de 10 → inválido
- **R\$10,00** → limite inferior válido
- **R\$500,00** → limite superior válido
- **R\$500,01** ou valor acima de 500 → inválido

Também seriam aceitáveis valores como 10, 11, 499,99, 500, 501, desde que a justificativa mostre raciocínio de borda. A aula de valor limite destaca testar exatamente sobre, imediatamente acima e imediatamente abaixo dos limites. O próprio cenário de delivery usa a faixa entre R\$10 e R\$500.

- D. Explique a importância de registrar corretamente os elementos de um caso de teste para a organização e clareza do processo de teste.

Resposta esperada:

- **ID:** permite rastreamento e organização;
- **objetivo:** deixa claro o que se quer verificar;
- **entradas:** especificam o dado usado no teste;
- **pré-condições:** garantem contexto correto de execução;
- **passos/procedimento:** tornam a execução reproduzível;
- **resultado esperado:** define critério claro de aprovação/reprovação.